PAP 2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Dieses Dokument ist von mir (FEZ) selbstständig geschreiben und verfasst worden. Viele Funktionen sind von anderen übernommen und abgewandelt worden. Da der Code in VS Code geschrieben wird, wird teilweise auf den MS Copilot (K.I. Modell) zurückgegriffen. Dieser wird unterstützend und nicht generierend benutzt. Ich entschuldige mit hiermit auch formal für das unglaublich schlimme Denglisch in den Kommentaren und der Dokumentation.

Vorlage zum bestimmen aller im Praktikum benötigten Formel und intigriertem Latex Export für eine effizientere Nutzung.

Diese Version ist noch nicht aktiell. Daher führen viele Überschriften in Leere Kapitel. (Stand 07.03.2026)

Inhaltsverzeichnis für den Papulator

- Definition der Versuchsvariablen
- Import aller genutzen Libaries
- Funktionen der Fehlerrechnung
  - Runden signifikanter Stellen
  - Gaussische Fehlerfortpflanzung
  - Berechnung von Messwert und Fehler
  - Signifikante Standartabweichung zweier Messwerte
- Messdatenimporte
  - Patterns
  - Import von Experimentdaten
- Latex Export
  - Export von Gleichungen
  - Export von Einzelergebnissen
  - Export von Messreihen (Tabelle))
- Calc_Everything
- Plotting
- Ergebnisse der Aufgaben

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

In [398…

```
versuchsnummer = "232"
versuchsname = "Michelson-Interferometer"
aufgabe = "0z"

# Wichtige Konstanten
# Heidelberger Ortsgravitationsbeschleunigung in m/s^2
g_hd = 9.80984
err_g_hd = 0.00002

# Farben in hex
gruen="#1b722f"
hellgruen = "#1ba97f"
rot = "#b92d3a"
weinrot="#721c24"
```

```
    function = sp.separatevars(function)

    if print_formula:
        print("gegebene Funktion:")
        function_to_latex(function)

        print("Formel des absoluten Fehlers der gegebenen Funktion:")
        function_to_latex(absolut_err)

        print("Formel des relativen Fehlers der gegebenen Funktion:")
        function_to_latex(relativ_err)

    return(results)


def calc2(function, params,data, params_without_error = [], p_latex:bool = True):
    length = 1
    for e in data:
        if(np.shape(np.array(e)) != ()):
            length = np.shape(np.array(e))[0]
            break
    data_2d = np.zeros((length,len(data)))
    for i in range (0,len(data)):
        data_2d[:,i] = data[i]
    return(calc_everything(function,params, data_2d,params_without_error = params_without_error, print_formula=p_latex)
```

Plotting

Ergebnisse der Aufgaben

Aufgabe 1

In [412…

```
# ---------------------------------
#   Messwerte der Tabelle 1.1
# ---------------------------------

# Weißlichtposition mm in m
x_0 = 2e-3

# Raumtemperatur °C in °K
temp_raum = 23.6 + 273.15
err_temp_raum = 0.5

# Position der Start- und Endpunkte der Messuhr in micrometer in meter
pos_start = np.array([100, 506, 1102, 1401, 1842])*1e-6
pos_end =np.array([3111, 3572, 4173, 4475, 4822])*1e-6
err_pos = 9e-6

# Anzahl der Impulse
n_imp = np.array([11045, 11170, 11183, 11189, 11099])
err_n_imp = np.std(n_imp)

# Herstellerangaben nm in m
lit_wellLang = 532e-9
err_lit_wellLang = 1e-9

# Normalbedinungen in Kelvin bzw. Pa
temp_0 = 273.15
p_0 = 101325

# Länge der Küvette von mm in m
ak = 50e-3
err_ak = 0.05e-3
```

In [413…

```
# ---------------------------------
#   Bestimmung der Wellenlänge
# ---------------------------------

pos_start_sym, pos_end_sym, steig_sym = sp.symbols(r'{s_a}, {s_e}, {m}')
params = [pos_start_sym, pos_end_sym, steig_sym]

wellenLange = 2 * (pos_end_sym - pos_start_sym) / steig_sym

values = np.array([
    pos_start, [err_pos] * len(pos_start),
    pos_end, [err_pos] * len(pos_end),
```

```
    n_imp, [err_n_imp] * len(n_imp)
])

lambda_res = calc2(wellenLange, params, values, [] , False)
lambda_val = lambda_res[:,0]
lambda_err = lambda_res[:,1]

# ---------------------------------
#   Export als Latex-Tabelle
# ---------------------------------

# Anpassen der Digids; muss variiert werden, da roudn_sig_digs hier nicht klappt :(
def pm(val, err, digits=2):
    return [f"{v:.{digits}f} \\pm {e:.{digits}f}" for v, e in zip(val, err)]

df = pd.DataFrame({
    r'Messung': np.arange(1, 6),
    r'\lambda \mathrm{[10^{-9} m]}': pm(lambda_val*1e9, lambda_err*1e9, digits=0),
    r'\text{Standardabweichung} \simga': abs(lambda_val - lit_wellLang)/np.sqrt(lambda_err**2 + err_lit_wellLang**2)
})

latex_table = df.to_latex(
    index=False,
    escape=False,
    column_format="c | C || C",
    caption="Wellenlängen der einzelmessungen",
    label="tab:wellenlangen",
)

print(latex_table)
```

```
\begin{table}
\caption{Wellenlängen der einzelmessungen}
\label{tab:wellenlangen}
\begin{tabular}{c | C || C}
\toprule
Messung & \lambda \mathrm{[10^{-9} m]} & \text{Standardabweichung} \simga \\
\midrule
1 & 545 \pm 4 & 3.531728 \\
2 & 549 \pm 4 & 4.562606 \\
3 & 549 \pm 4 & 4.635252 \\
4 & 549 \pm 4 & 4.701490 \\
5 & 537 \pm 4 & 1.348617 \\
\bottomrule
\end{tabular}
\end{table}
```

In [414…

```
# ---------------------------------
#   Finale Wellenlänge
# ---------------------------------

avg_wellLang = np.mean(lambda_val)
err_avg_wellLang = (1/len(lambda_val) * np.std(lambda_val))

print(f"Finale Wellenlänge: {avg_wellLang * 1e9:.1f} \\pm {err_avg_wellLang * 1e9:.1f} e-9 m")

abweichung = abs(avg_wellLang - lit_wellLang)/np.sqrt(err_avg_wellLang**2 + err_lit_wellLang**2)

print(f"Signifikante Abweichung: {abweichung:.2f}")
```

```
Finale Wellenlänge: 546.0 \pm 1.0 e-9 m
Signifikante Abweichung: 10.13
```

Aufgabe 2

In [415…

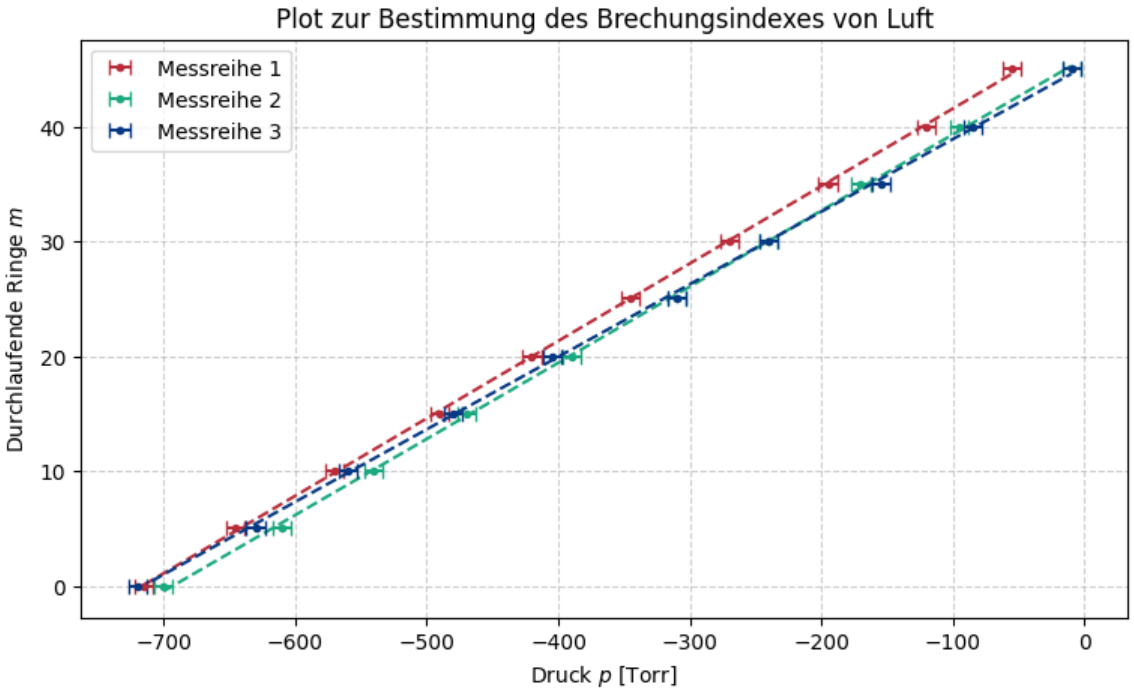
```
# ---------------------------------
#   Messwerte der Tabelle 2.1
# ---------------------------------

# Anzahl der Durchlaufenen Ringe (0 bis 45 in 5er Schritten)
n_ring = np.array([])
for i in range(0,46, 5):
    n_ring = np.append(n_ring, i)

# Drücke der Messungen in Torr
p_1 = np.array([715, 645, 570, 490, 420, 345, 270, 195, 120, 55]) * (-1)
p_2 = np.array([700, 610, 540, 470, 390, 310, 240, 170, 95, 10]) * (-1)
p_3 = np.array([720, 630, 560, 480, 405, 310, 240, 155, 85, 10]) * (-1)

err_p_abl = 5
err_p_sys = 0.006 * 800
err_p = np.sqrt(err_p_abl**2 + err_p_sys**2)
```

In [416…

```
aufgabe = "2"
pfad = "../img/"
# ---------------------------------
```

```python
#   Plotting
# ---------------------------------

def linear(x,a,b):
    return a*x+b

# Jedem Plot ein Label geben. Reihenfolge muss identisch zu dem Format oben sein!
labels = [
    "Messreihe 1",
    "Messreihe 2",
    "Messreihe 3",
]

# Farben müssen halt mega cool aussehen, sonst ist das Protokoll nur mid
colors = [
    rot,
    hellgruen,
    blau,
    ]

drucks = [p_1, p_2, p_3]

plt.figure(figsize=(9,5))

for i in range(len(drucks)):
    plt.errorbar(
        x = drucks[i],
        y = n_ring,
        xerr=err_p,
        fmt='.',
        color=colors[i],
        capsize=3.5,
        label=labels[i]
        )

    popt, pcov = curve_fit(linear, drucks[i], n_ring)

    steig = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
    absch = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

    print(f"Steigung {labels[i]}: {steig[2]} \n", f"y-Achsenabschnitt: {absch[2]}")

    plt.plot(
        drucks[i],
        linear(drucks[i], *popt),
        color=colors[i],
        linestyle='--'
        )

# ------------------------
#   Erstellung des Plots
# ------------------------

plt.xlabel("Druck $p \\ [\\mathrm{Torr}]$")
plt.ylabel("Durchlaufende Ringe $m$")
plt.title('Plot zur Bestimmung des Brechungsindexes von Luft')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()

plt.savefig(f'{pfad}ringe_gegen_druck{aufgabe}.pdf', dpi=300)
plt.show()
```

```
Steigung Messreihe 1: (6.75 \pm 0.03) e-2
 y-Achsenabschnitt: 48 \pm 0.14
Steigung Messreihe 2: (6.63 \pm 0.05) e-2
 y-Achsenabschnitt: 46 \pm 0.21
Steigung Messreihe 3: (6.32 \pm 0.05) e-2
 y-Achsenabschnitt: 45 \pm 0.23
```

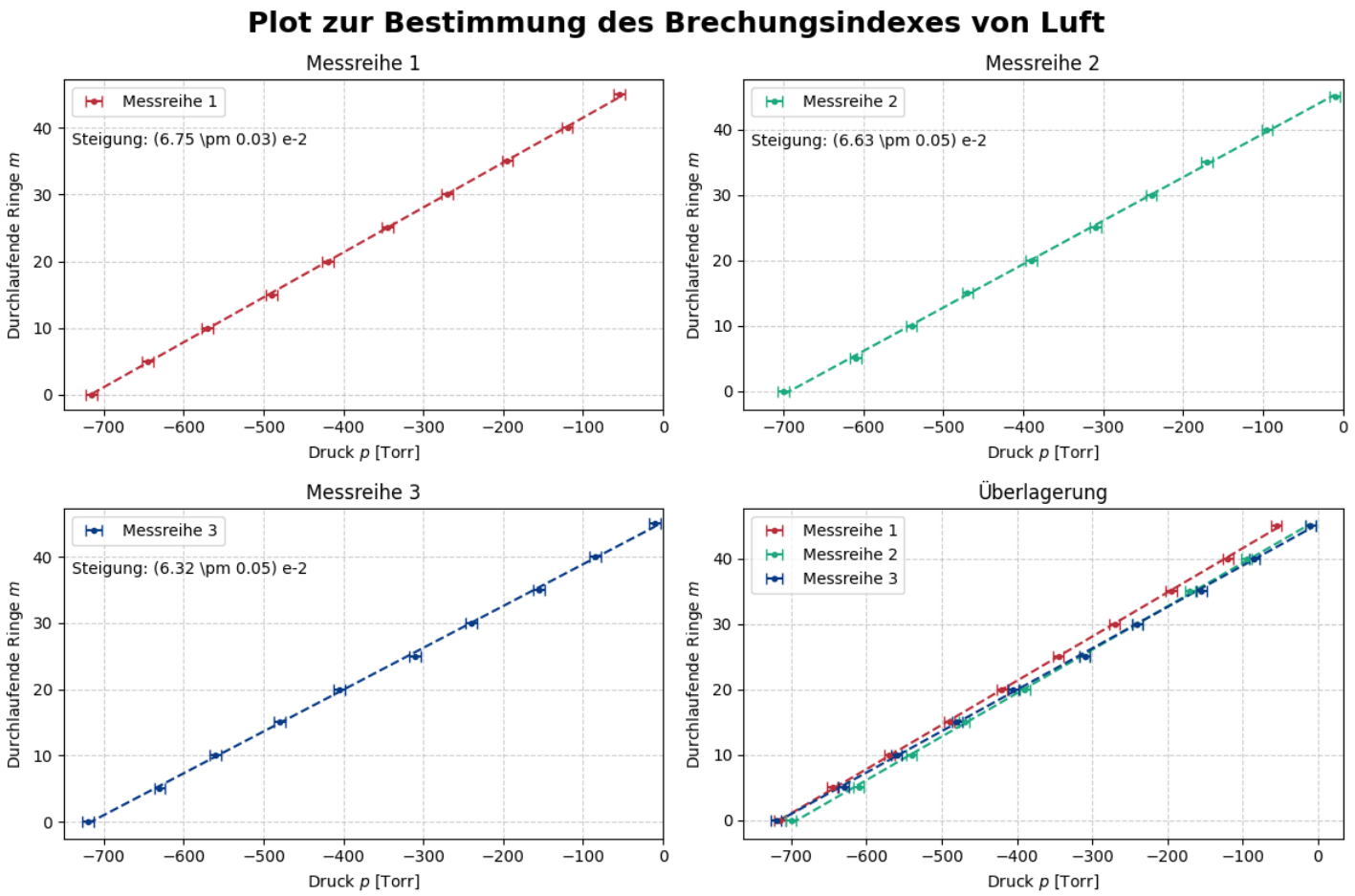


In [417…

```
aufgabe = "2"
pfad = "../img/"

def linear(x, a, b):
    return a * x + b

labels = [
    "Messreihe 1",
    "Messreihe 2",
    "Messreihe 3",
]

colors = [
    rot,
    hellgruen,
    blau,
]

drucks = [p_1, p_2, p_3]

# 2x2 Grid
fig, axs = plt.subplots(2, 2, figsize=(12, 8))
axs = axs.flatten()

# ------------------------
#   Einzelplots
# ------------------------

steigungen_torr = []

for i in range(3):
    ax = axs[i]

    ax.errorbar(
        x=drucks[i],
        y=n_ring,
        xerr=err_p,
        fmt='.',
        color=colors[i],
        capsize=3.5,
        label=labels[i]
    )

    popt, pcov = curve_fit(linear, drucks[i], n_ring)

    steig = round_sig_digs(popt[0], np.sqrt(pcov[0, 0]))
    absch = round_sig_digs(popt[1], np.sqrt(pcov[1, 1]))

    print(f"Steigung {labels[i]}: {steig[2]}")
    print(f"y-Achsenabschnitt {labels[i]}: {absch[2]}\n")

    ax.plot(
        drucks[i],
        linear(drucks[i], *popt),
        color=colors[i],
        linestyle='--'
    )

    ax.set_title(labels[i])
```

```python
    ax.set_xlabel("Druck $p \\ [\\mathrm{Torr}]$")
    ax.set_ylabel("Durchlaufende Ringe $m$")
    ax.grid(True, linestyle="--", alpha=0.6)
    ax.set_xlim(-750, 0)
    ax.text(-740, 37.5, f"Steigung: {steig[2]}")
    ax.legend()

    steigungen_torr.append(steig)

# ------------------------
#   Überlagerungsplot
# ------------------------
ax = axs[3]

for i in range(len(drucks)):
    ax.errorbar(
        x=drucks[i],
        y=n_ring,
        xerr=err_p,
        fmt='.',
        color=colors[i],
        capsize=3.5,
        label=labels[i]
    )

    popt, _ = curve_fit(linear, drucks[i], n_ring)

    ax.plot(
        drucks[i],
        linear(drucks[i], *popt),
        color=colors[i],
        linestyle='--'
    )

ax.set_title("Überlagerung")
ax.set_xlabel("Druck $p \\ [\\mathrm{Torr}]$")
ax.set_ylabel("Durchlaufende Ringe $m$")
ax.grid(True, linestyle="--", alpha=0.6)
ax.legend()

# Gesamttitel
fig.suptitle('Plot zur Bestimmung des Brechungsindexes von Luft', fontsize=18, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{pfad}ringe_gegen_druck_SUBPOLLTING_{aufgabe}.pdf', dpi=300)
plt.show()
```

```
Steigung Messreihe 1: (6.75 \pm 0.03) e-2
y-Achsenabschnitt Messreihe 1: 48 \pm 0.14

Steigung Messreihe 2: (6.63 \pm 0.05) e-2
y-Achsenabschnitt Messreihe 2: 46 \pm 0.21

Steigung Messreihe 3: (6.32 \pm 0.05) e-2
y-Achsenabschnitt Messreihe 3: 45 \pm 0.23
```

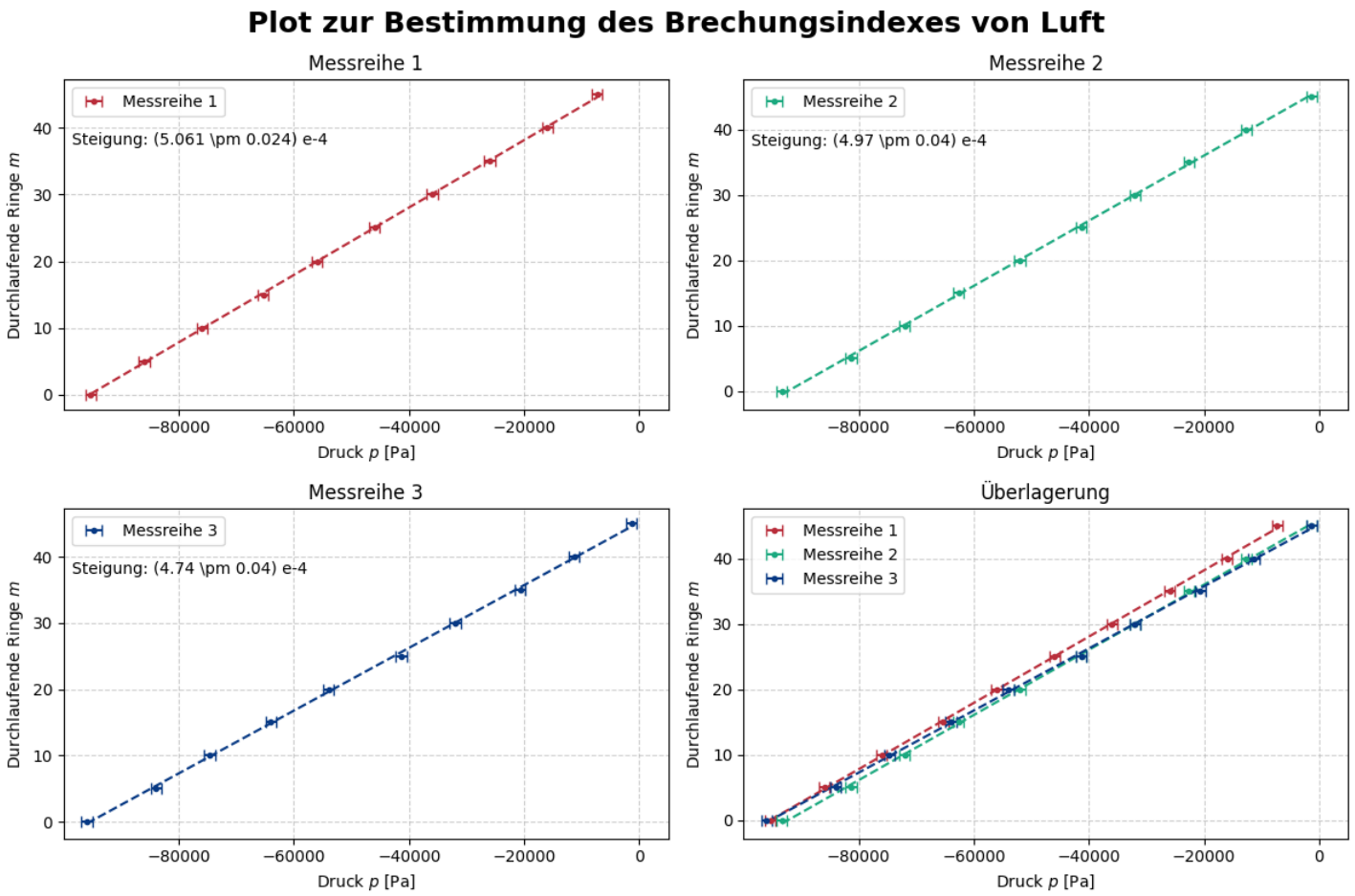


In [418…

```
aufgabe = "2"
pfad = "../img/"

def linear(x, a, b):
    return a * x + b

labels = [
    "Messreihe 1",
    "Messreihe 2",
    "Messreihe 3",
]

colors = [
    rot,
    hellgruen,
    blau,
]

drucks = np.array([p_1, p_2, p_3]) * 133.322  # Umrechnung von Torr in Pa

# 2x2 Subplots erstellen
fig, axs = plt.subplots(2, 2, figsize=(12, 8))
axs = axs.flatten()

# ------------------------
#   Einzelplots
# ------------------------

steigungen_pa = []

for i in range(3):
    ax = axs[i]

    ax.errorbar(
        x=drucks[i],
        y=n_ring,
        xerr=err_p * 133.322,
        fmt='.',
        color=colors[i],
        capsize=3.5,
        label=labels[i]
    )

    popt, pcov = curve_fit(linear, drucks[i], n_ring)

    steig = round_sig_digs(popt[0], np.sqrt(pcov[0, 0]))
    absch = round_sig_digs(popt[1], np.sqrt(pcov[1, 1]))

    print(f"Steigung {labels[i]}: {steig[2]}")
```

```python
    print(f"y-Achsenabschnitt {labels[i]}: {absch[2]}\n")

    ax.plot(
        drucks[i],
        linear(drucks[i], *popt),
        color=colors[i],
        linestyle='--'
    )

    ax.set_title(labels[i])
    ax.set_xlabel("Druck $p \\ [\\mathrm{Pa}]$")
    ax.set_ylabel("Durchlaufende Ringe $m$")
    ax.grid(True, linestyle="--", alpha=0.6)
    ax.set_xlim(-750 * 133.322, 5000)
    ax.text(-740 * 133.322, 37.5, f"Steigung: {steig[2]}")
    ax.legend()

    steigungen_pa.append(steig)

# ------------------------
#   Überlagerungsplot
# ------------------------
ax = axs[3]

for i in range(len(drucks)):
    ax.errorbar(
        x=drucks[i],  # Umrechnung von Torr in Pa
        y=n_ring,
        xerr=err_p * 133.322,
        fmt='.',
        color=colors[i],
        capsize=3.5,
        label=labels[i]
    )

    popt, _ = curve_fit(linear, drucks[i], n_ring)

    ax.plot(
        drucks[i],
        linear(drucks[i], *popt),
        color=colors[i],
        linestyle='--'
    )

ax.set_title("Überlagerung")
ax.set_xlabel("Druck $p \\ [\\mathrm{Pa}]$")
ax.set_ylabel("Durchlaufende Ringe $m$")
ax.grid(True, linestyle="--", alpha=0.6)
ax.set_xlim(-750 * 133.322, 5000)
ax.legend()

# Gesamttitel
fig.suptitle('Plot zur Bestimmung des Brechungsindexes von Luft', fontsize=18, fontweight='bold')

plt.tight_layout()
plt.savefig(f'{pfad}ringe_gegen_druck_SUBPOLLTING_{aufgabe}.pdf', dpi=300)
plt.show()
```

```
Steigung Messreihe 1: (5.061 \pm 0.024) e-4
y-Achsenabschnitt Messreihe 1: 48 \pm 0.14

Steigung Messreihe 2: (4.97 \pm 0.04) e-4
y-Achsenabschnitt Messreihe 2: 46 \pm 0.21

Steigung Messreihe 3: (4.74 \pm 0.04) e-4
y-Achsenabschnitt Messreihe 3: 45 \pm 0.23
```



In [419…

```python
# ------------------------
#   Mittlere Steigung
# ------------------------

for i in range(len(steigungen_torr)):
    print(f"Steigung {labels[i]}: {steigungen_torr[i][2]}")

avg_steig = np.mean([s[0] for s in steigungen_torr])
avg_steig = float(avg_steig)
err_avg_steig = np.sqrt(np.sum([s[1]**2 for s in steigungen_torr])) / len(steigungen_torr)
err_avg_steig = float(err_avg_steig)
print(f"Mittlere Steigung: ({avg_steig * 1e2:.2f} \\pm {err_avg_steig * 1e2:.2f}) e-2 1/Torr")
```

```
Steigung Messreihe 1: (6.75 \pm 0.03) e-2
Steigung Messreihe 2: (6.63 \pm 0.05) e-2
Steigung Messreihe 3: (6.32 \pm 0.05) e-2
Mittlere Steigung: (6.57 \pm 0.03) e-2 1/Torr
```

In [420…

```python
# 1 Torr = 133.322 Pa

avg_steig_pa = np.mean([s[0] for s in steigungen_pa])
avg_steig_pa = float(avg_steig_pa)
err_avg_steig_pa = np.sqrt(np.sum([s[1]**2 for s in steigungen_pa])) / len(steigungen_pa)
err_avg_steig_pa = float(err_avg_steig_pa)
print(f"Mittlere Steigung: ({avg_steig_pa * 1e4:.1f} \\pm {err_avg_steig_pa * 1e4:.1f}) \\times 10^{{-4}} \\, \\mathrm{
```

```
Mittlere Steigung: (4.9 \pm 0.0) \times 10^{-4} \, \mathrm{1/Pa}
```

In [421…

```python
# ------------------------------------------------------------------------
#   Bestimmung des Brechungsindexes von Luft
# ------------------------------------------------------------------------

lam_sym, lange_kuvette, p_0_sym, temp_0_sym, raum_temp_sym, DieseSteigungDa = sp.symbols(r'\lambda, a, {p_0}, {T_0}, T,
params = [lam_sym, lange_kuvette, raum_temp_sym, DieseSteigungDa, p_0_sym, temp_0_sym]
n_0 = 1 + DieseSteigungDa * (lam_sym / (2 * lange_kuvette)) * (raum_temp_sym * p_0_sym  / temp_0_sym)

values = np.array([
    avg_wellLang, err_avg_wellLang,
    ak, err_ak,
    temp_raum, err_temp_raum,
    avg_steig_pa, err_avg_steig_pa,
    p_0,
    temp_0
])

print(f'Wellenlänge: {avg_wellLang *1e9:.0f} \\pm {err_avg_wellLang *1e9:.0f} nm')
print(f'Küvettenlänge: {ak*1e3:.2f} \\pm {err_ak*1e3:.2f} mm')
print(f'Raumtemperatur: {temp_raum} \\pm {err_temp_raum} °K')
print(f'Abgeleitete Steigung: ({avg_steig_pa * 1e4:.1f} \\pm {err_avg_steig_pa * 1e4:.1f}) \\times 10^{{-4}} \\, \\math
print(f'Normalbedingungen: {p_0} Pa, {temp_0} °K\n')
```

```python
n_luft_norm = calc2(n_0, params, values, params_without_error=[p_0_sym, temp_0_sym], p_latex=False)
val_n_luft = n_luft_norm[:,0]
err_n_luft = n_luft_norm[:,1]

print(f"Brechungsindex von Luft: {val_n_luft[0]:.7f} \\pm {err_n_luft[0]:.7f}")


# Abweichungen
abw = abs(val_n_luft - 1.00028) / err_n_luft
prozentAbw = abs(val_n_luft - 1.00028) / 1.00028 * 100
print(f'Die Standardabweichung beträgt: {abw[0]:.2f}')
print(f'Die prozentuale Abweichung beträgt: {prozentAbw[0]:.4f} %')
```

```
Wellenlänge: 546 \pm 1 nm
Küvettenlänge: 50.00 \pm 0.05 mm
Raumtemperatur: 296.75 \pm 0.5 °K
Abgeleitete Steigung: (4.9 \pm 0.0) \times 10^{-4} \, \mathrm{1/Pa}
Normalbedingungen: 101325 Pa, 273.15 °K

Brechungsindex von Luft: 1.0002959 \pm 0.0000015
Die Standardabweichung beträgt: 10.94
Die prozentuale Abweichung beträgt: 0.0016 %
```

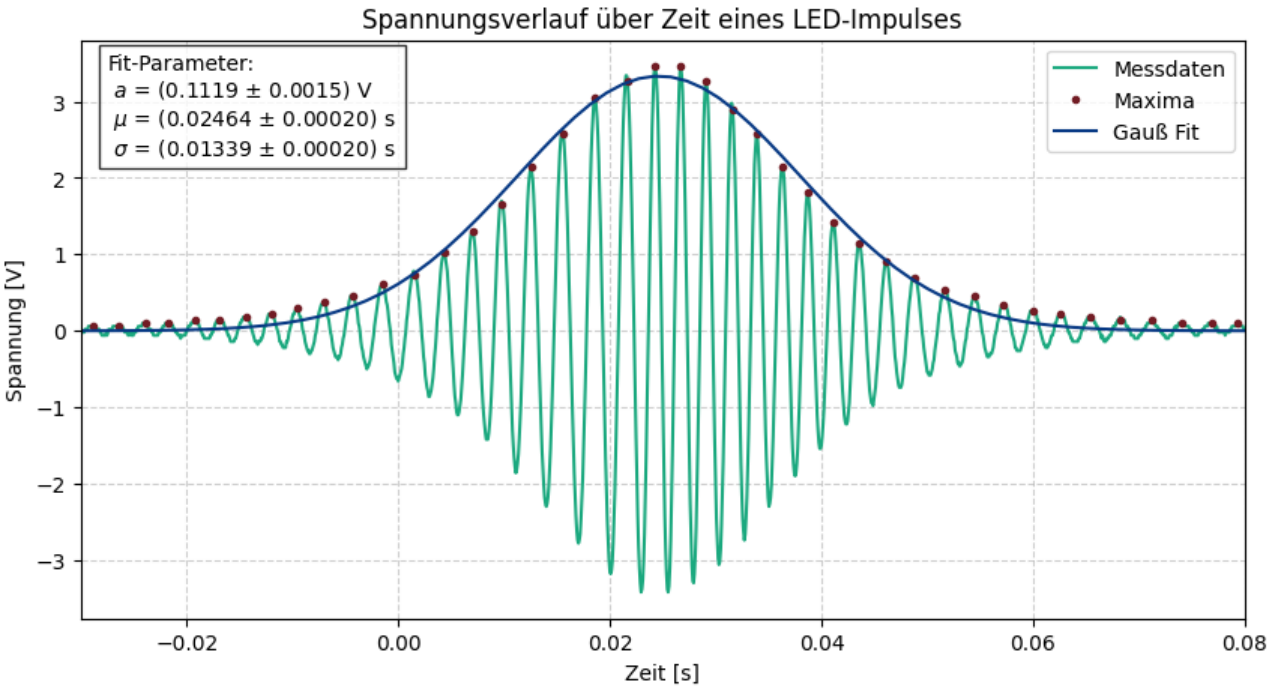
Aufgabe 3

In [448…

```python
# ---------------------------------------------------------------------------
#   Plotten des LED Spannungsverlaufs über Zeit und Gaußfit
# ---------------------------------------------------------------------------
# Oszi Messwerte
data = pd.read_csv('TEK00002.CSV', sep=',', decimal='.', skiprows=15)

ch1 = data['CH1']
time = data['TIME']

# Peaks finden
peakind = signal.find_peaks_cwt(ch1, np.arange(1,30),noise_perc=20)

#Fitfunktion Gauss
def fitFunc(t, a, mu, sig):
    return a/np.sqrt(2*np.pi)/sig*np.exp(-(t-mu)**2/(2*sig**2))

popt, pcov= curve_fit(fitFunc, time[peakind], ch1[peakind])

print(f'a = {popt[0]:.4f} \\pm {np.sqrt(pcov[0,0]):.4f} V')
print(f'mu = {popt[1]:.5f} \\pm {np.sqrt(pcov[1,1]):.5f} s')
print(f'sigma = {popt[2]:.5f} \\pm {np.sqrt(pcov[2,2]):.5f} s')

x=np.linspace(-0.05,0.1,100)

# Actual Plotting
plt.figure(figsize=(10,5))

# Signal
plt.plot(time,
         ch1,
         label='Messdaten',
         color= hellgruen
         )

# Extrema
plt.plot(time[peakind],
         ch1[peakind],
         marker='.',
         linewidth=0,
        color=weinrot,
        label='Maxima'
        )

# Fit
plt.plot(x,
         fitFunc(x, *popt),
         color=blau,
         label='Gauß Fit'
         )

plt.xlim(-0.03, 0.08)
plt.xlabel('Zeit [s]')
plt.ylabel('Spannung [V]')
plt.title('Spannungsverlauf über Zeit eines LED-Impulses')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()

plt.text(-0.0275, 2.3, f"Fit-Parameter:\n $a$ = ({popt[0]:.4f} $\\pm$ {np.sqrt(pcov[0,0]):.4f}) V\n $\\mu$ = ({popt[1]:

plt.savefig('../img/spannung_zeitverlauf.pdf', dpi=300)
```

```
plt.show()
```

```
a = 0.1119 \pm 0.0015 V
mu = 0.02464 \pm 0.00020 s
sigma = 0.01339 \pm 0.00020 s
```



In [456…

```
# ---------------------------------------------------------------------
#   Bestimmung der Halbwertszeit und Kohärenzlänge der LED
# ---------------------------------------------------------------------

# Berechnung der Halbwertszeit
amp, mu, sig = popt
err_amp, err_mu, err_sig = np.sqrt(np.diag(pcov))

halbwertszeit = 2 * sig * np.sqrt(2 * np.log(2))
err_halbwertszeit = 2 * err_sig * np.sqrt(2 * np.log(2))

print(f'Die Halbwertszeit beträgt: ({halbwertszeit:.4f} \\pm {err_halbwertszeit:.4f}) s')

# Berechnung der Kohärenzlänge
# Geschwindigkeit des Motors von mm/s in m/s
v_motor = 0.1e-3

koharenzlange = 2* v_motor * halbwertszeit
err_koharenzlange = 2* v_motor * err_halbwertszeit

print(f'Die Kohärenzlänge beträgt: ({koharenzlange * 1e6:.2f} \\pm {err_koharenzlange * 1e6:.2f})*e6 m')
```

```
Die Halbwertszeit beträgt: (0.0315 \pm 0.0005) s
Die Kohärenzlänge beträgt: (6.30 \pm 0.10)*e6 m
```